

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1003	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	P. MUNI KARTHIK	Reg. No.:	192425061
Date:		Duration:	60 minutes

Code of Conduct

I P. MUNI KARTHIK/192425061 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: P. MUNI KARTHIK

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.

Problem Statement

The **Smart Warehouse Management System** is designed to automate warehouse operations such as inventory management, barcode scanning, supplier management, stock tracking, and report generation. The system helps reduce manual work, improve inventory accuracy, and enhance warehouse efficiency.

System Objectives

- Automate warehouse operations.
 - Maintain accurate inventory records.
 - Monitor stock movement in real time.
 - Generate inventory reports.
 - Improve warehouse productivity.
 - Reduce human errors.
 - Enable secure user access.
-
- Analyze the functional and non-functional requirements that influence the software architecture.

Functional Requirements

- User Login and Authentication
- User Role Management (Admin, Manager, Staff)
- Add New Stock
- Remove Stock
- Update Inventory
- Barcode Scanning
- Supplier Management
- Inventory Monitoring
- Generate Reports
- Product Search
- Dashboard for Warehouse Manager

Non-Functional Requirements

- Fast system response
 - Secure login and data protection
 - Easy maintenance
 - High availability
 - Reliable data storage
 - Scalable architecture
 - User-friendly interface
-
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

Quality Attributes

Quality Attribute	Description
Scalability	Supports increasing warehouse data and users
Maintainability	Easy to update and modify modules
Reliability	Accurate inventory without data loss
Availability	System available whenever required
Performance	Fast barcode scanning and report generation
Security	User authentication and access control

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).

Selected Architecture

Layered Architecture (Three-Tier Architecture)

Layers

1. Presentation Layer
2. Business Logic Layer
3. Data Layer

- Justify why the selected architecture is the most suitable for the given problem.

Justification

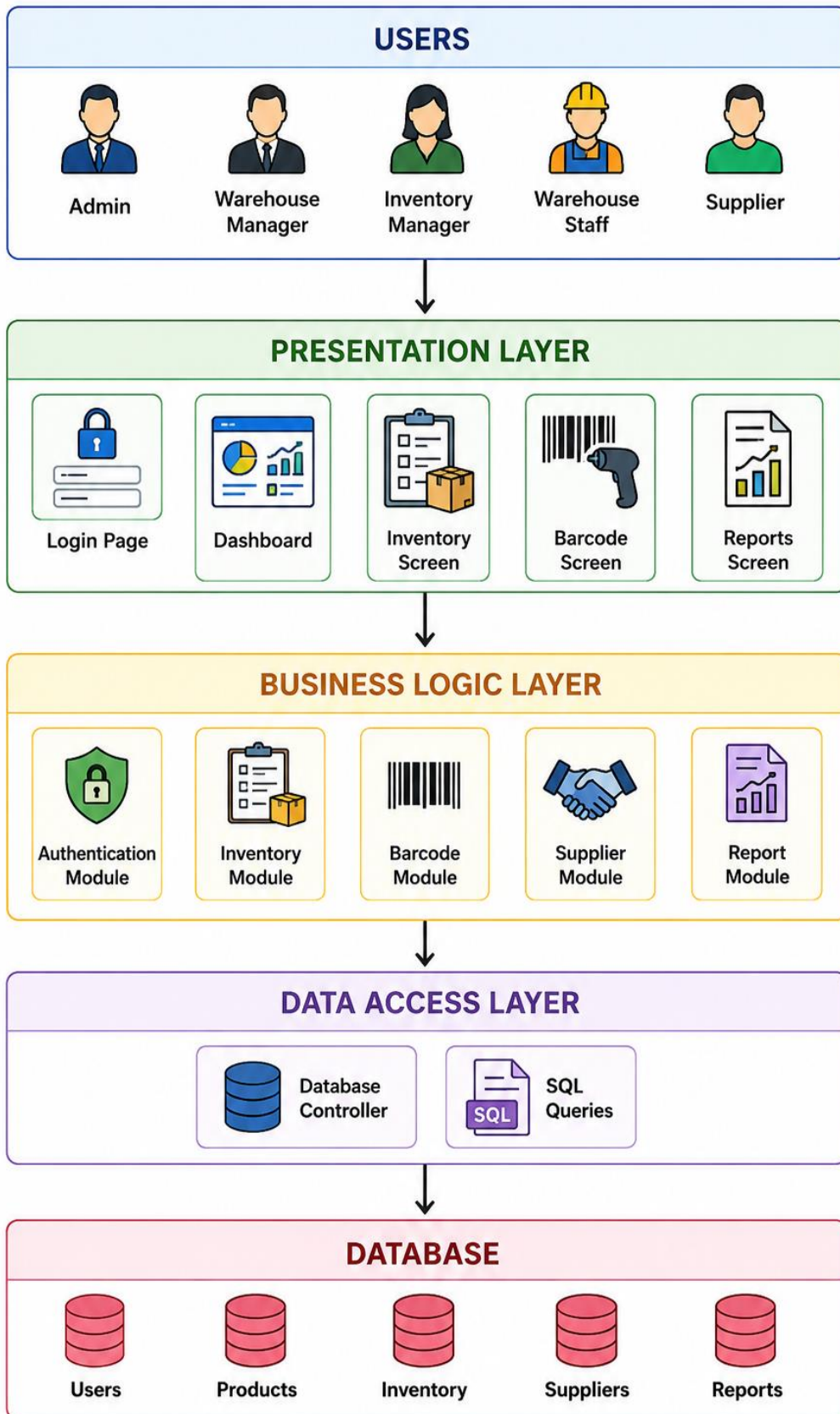
Layered Architecture is suitable because:

- Separates application into independent layers.
- Easier to maintain.
- Easy to test.
- Supports future enhancements.
- Improves code organization.
- Secure communication between layers.
- Commonly used in inventory management systems.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

SMART WAREHOUSE MANAGEMENT SYSTEM



4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

1. Presentation Layer

Responsibilities

- Displays user interface.
- Accepts user input.
- Shows reports and inventory information.

Components

- Login Page
 - Dashboard
 - Inventory Management Screen
 - Barcode Scanner Screen
 - Report Screen
- Explain how the components communicate and exchange data.

2. Business Logic Layer

Responsibilities

- Processes business rules.
- Validates user input.
- Controls inventory operations.
- Generates reports.

Modules

- Authentication Module
- Inventory Module
- Barcode Module
- Supplier Module
- Report Module

3. Data Access Layer

Responsibilities

- Connects application with database.
- Executes SQL queries.
- Retrieves and stores data.

4. Database

Stores

- User Information
 - Product Details
 - Inventory Records
 - Supplier Information
 - Report History
-
- Illustrate the overall workflow of the system.

Workflow

1. User logs into the system.
2. Login request goes to Authentication Module.
3. Authentication Module verifies credentials.
4. Database validates user.
5. Dashboard is displayed.
6. User performs inventory operations.
7. Business Logic updates inventory.
8. Database stores changes.
9. Reports are generated whenever requested.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

Scalability

- New warehouse branches can be added easily.
- Supports large inventory databases.
- Handles increasing users without changing architecture.

Security

- Username and password authentication.
- Role-based access control.
- Database access only through business layer.
- Secure data storage.

Performance

- Fast inventory search.
- Quick barcode scanning.
- Efficient SQL queries.
- Fast report generation.

Reliability

- Accurate inventory updates.
- Prevents duplicate entries.
- Consistent database transactions.
- Error handling during stock operations.

Maintainability

- Each layer is independent.
- Easy to modify individual modules.
- New features can be added without affecting existing modules.

Availability

- Database backup support.
- Continuous warehouse access.
- Supports multiple users simultaneously.

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.

Why Layered Architecture?

The Layered Architecture separates the application into Presentation, Business Logic, and Data Access layers. This separation improves code organization, simplifies debugging, and allows independent development and testing of each layer.

- Discuss the trade-offs considered while selecting the architecture.

Trade-offs Considered

Advantages	Disadvantages
Easy maintenance	Slightly slower due to multiple layers
Better security	More components to manage
High reusability	Extra communication between layers
Easier testing	Initial development takes more time

- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Future Enhancements

The architecture supports future improvements such as:

- AI-based inventory prediction
- IoT sensor integration
- Cloud database migration
- Mobile application support
- QR code scanning
- Real-time notifications
- Analytics dashboard
- REST API integration
- RFID-based warehouse tracking

Long-Term Maintainability

The selected architecture ensures:

- Easy software updates.
- Independent module development.
- Simple debugging.
- Better testing.
- Easy integration with third-party systems.
- High scalability for future warehouse expansion.

Conclusion

The **Layered (Three-Tier) Architecture** is the most suitable choice for the **Smart Warehouse Management System** because it provides a clear separation of concerns, improves maintainability, enhances security, and supports scalability. By dividing the application into Presentation, Business Logic, and Data layers, the system can efficiently manage inventory, barcode scanning, supplier management, and reporting while remaining flexible for future enhancements such as cloud integration, IoT devices, and AI-based analytics. This architecture ensures reliable performance, long-term maintainability, and the ability to grow with evolving warehouse requirements.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25